

Università di Padova

DIPARTIMENTO DI MATEMATICA “TULLIO LEVI-CIVITA”

CORSO DI LAUREA IN INFORMATICA



Verifiche di cybersicurezza per dispositivi
embedded

Tesi di Laurea

Laureando

Nenad Radulovic

Matricola 2101059

Relatore

Prof. Tullio Vardanega

ANNO ACCADEMICO 2025-2026

[da scrivere]

Ringraziamenti

[da scrivere]

Padova, settembre 2026

Nenad Radulovic

Sommario

Nell'ambito dello sviluppo di sistemi *embedded*, la verifica della corretta implementazione dei protocolli di comunicazione rappresenta un aspetto cruciale per garantirne l'affidabilità, in particolare nei settori *automotive* e industriale.

La presente tesi descrive le attività svolte durante il mio *stage* universitario, incentrato sullo sviluppo di un *framework* di *test* per dispositivi *embedded* e sulla realizzazione di *test* per la verifica di specifici protocolli di comunicazione.

Durante il primo periodo, di diverse settimane, mi sono dedicato allo sviluppo, in linguaggio Python, di un *framework* per l'esecuzione automatizzata di *test*, che ha comportato sia il *refactoring* di *test* preesistenti sia la progettazione e la scrittura di nuovi casi di *test*. La maggior parte dei *test* realizzati era volta a verificare la comunicazione su canale CAN (*Controller Area Network*) e le sessioni diagnostiche UDS (*Unified Diagnostic Services*) su CAN, protocollo largamente diffuso nella diagnostica *automotive*.

Successivamente, ho esteso il progetto includendo la comunicazione Modbus su interfaccia UART, per la quale ho sviluppato *test* volti a verificare quali *function code* risultassero eseguibili dal dispositivo e su quali indirizzi. Per rendere possibile questa verifica è stato necessario, in uno stadio preliminare, programmare in linguaggio C una scheda Nucleo-H533RE, affinché fosse in grado di rispondere alle richieste Modbus e di implementare correttamente i *function code* previsti dal protocollo.

Nella tesi approfondisco inoltre le principali tecnologie e gli strumenti impiegati, discuto l'utilità dei *test* sviluppati e illustro le motivazioni alla base delle soluzioni progettuali adottate. Una sezione conclusiva è infine dedicata alle competenze acquisite durante l'esperienza, tra cui : competenze di programmazione in Python e in C, applicazione in ambito reale di concetti di progettazione del *software* e acquisizione di conoscenze sui protocolli CAN, UDS e Modbus.

Questo documento è formato dai seguenti capitoli:

- **Capitolo 1: Il contesto aziendale:** in questo capitolo introduco Bluewind SRL, l'azienda presso la quale ho svolto il mio *stage*. Ne descrivo l'organizzazione, il settore in cui operano e il loro approccio all'innovazione.
- **Capitolo 2: L'origine e gli scopi del tirocinio:** in questo capitolo spiego quali motivazioni hanno spinto l'azienda a ideare questa proposta di tirocinio e quali obiettivi si sono posti di raggiungere con esso.
- **Capitolo 3: Lo sviluppo del progetto:** in questo capitolo spiego le fasi di analisi, progettazione, sviluppo e verifica del progetto al quale ho lavorato, mostrando come ho deciso di interpretare e risolvere i problemi che ho incontrato.
- **Capitolo 4: Valutazione retrospettiva:** in questo capitolo faccio una valutazione retrospettiva sui risultati ottenuti, confrontandoli con gli obiettivi posti inizialmente. Parlo infine delle competenze che ho sviluppato durante questa esperienza.

Convenzioni tipografiche

Seguono le convenzioni tipografiche che ho adottato durante la scrittura di questo documento:

- Vengono evidenziati in *corsivo* tutti i termini in lingua diversa dall'italiano.
- Vengono evidenziati in **grassetto** i termini su cui si vuole far riporre maggiore attenzione.
- Sarà presente il simbolo G in pedice a tutti i termini non di uso comune e agli acronimi, la cui definizione viene approfondita all'interno del glossario consultabile alla fine del documento. Questa convenzione verrà applicata solo alla prima occorrenza del termine.

Indice

1	Il contesto aziendale	1
1.1	Bluewind: aree di attività e mercati di riferimento	1
1.2	L'organizzazione aziendale e il metodo di lavoro	3
1.2.1	I ruoli	3
1.2.2	Il metodo <i>Agile</i>	5
1.3	I clienti	7
1.4	Lo <i>stack</i> tecnologico aziendale	9
1.4.1	Visione generale sull'adozione delle tecnologie	9
1.4.2	Gitlab	10
1.4.3	Servizi e strumenti proprietari	11
1.4.4	StrictDoc	11
1.4.5	Il linguaggio C	11
1.4.6	Il linguaggio Assembly	12
1.4.7	PC-lint	12
1.4.8	Strumenti proprietari di aziende <i>partner</i> come Infineon e ST	13
1.5	Propensione all'innovazione	13
2	L'origine e gli scopi del tirocinio	15
2.1	Gli <i>stage</i> all'interno del piano aziendale	15
2.2	La spinta normativa	16
2.2.1	Cyber Resilience Act	16
2.2.2	Regolamenti europei No. 155 e No. 156	17
2.2.3	<i>Standard</i> ETSI EN 303645	17

2.2.4	Regole MISRA	18
2.3	Il progetto e la sua finalità	19
2.4	Gli obiettivi del progetto	19
2.4.1	Obiettivi obbligatori	19
2.4.2	Obiettivi desiderabili	20
2.4.3	Obiettivi facoltativi	21
2.5	Le ambizioni personali	21
2.5.1	La scelta del tirocinio	21
2.5.2	Gli obiettivi personali	22
3	Lo sviluppo del progetto	24
3.1	Il metodo di lavoro	24
3.2	Le tecnologie utilizzate	26
3.2.1	Linguaggi e <i>framework</i>	26
3.2.2	Protocolli	27
3.2.3	Schede <i>hardware</i>	29
3.2.4	Strumenti proprietari esterni	29
3.3	Analisi	29
3.4	Progettazione	29
3.4.1	Introduzione	29
3.4.2	Le classi ‘ <i>Target</i> ’	29
3.4.3	Le classi ‘ <i>Test</i> ’	29
3.4.4	Creazione ed esecuzione	30
3.4.5	Persistenza	30
3.4.6	API	30
3.4.7	<i>Frontend</i>	30
3.5	Programmazione	30
3.6	Verifica e validazione	31
3.7	Resoconto del lavoro svolto	31
3.7.1	I risultati conseguiti	31
3.7.2	Migliorie possibili	31

4	Valutazione retrospettiva	32
4.1	Valutazione del raggiungimento degli obiettivi	32
4.2	La maturazione professionale	32
4.3	Confronto con il piano di studi	32
	Acronimi e abbreviazioni	i
	Glossario	ii
	Bibliografia	vi

Elenco delle figure

1.1	Rappresentazione grafica di un esempio di gerarchia dei ruoli all'interno dell'azienda.	5
1.2	Schema che rappresenta la differenza di interazioni tra i clienti in ambito <i>automotive</i> e quelli in ambito industriale generale	8
1.3	Schema che rappresenta l'interazione delle tecnologie all'interno dei processi aziendali.	10
3.1	Estratto del documento "Piano di lavoro" contenente la suddivisione delle attività per ogni settimana prevista dal tirocinio. . .	25
3.2	Schema che rappresenta i protocolli utilizzati per la comunicazione CAN all'interno della pila ISO-OSI.	28

Elenco delle tabelle

Elenco dei codici sorgenti

Capitolo 1

Il contesto aziendale

Nel seguente capitolo presento il contesto organizzativo e produttivo dell'azienda Bluewind SRL, presso la quale ho svolto la mia attività di tirocinio. Durante la mia permanenza in azienda ho avuto modo di discutere con diversi colleghi sulla loro realtà lavorativa, e mi sono informato quanto più possibile sul tipo di attività che svolgono. Questo capitolo contiene le informazioni sull'organizzazione, i metodi di lavoro e la cultura aziendale di Bluewind che sono riuscito ad apprendere durante quest'esperienza.

1.1 Bluewind: aree di attività e mercati di riferimento

Fondata nel 1998, Bluewind è un'azienda informatica specializzata nel campo dei sistemi *embedded* e della sicurezza informatica. Per *embedded* si intendono sistemi elettronici, dalle risorse limitate (sia come memoria che come potenza di calcolo), progettati con soluzioni *hardware* e *software* su misura per svolgere in modo efficiente una funzione dedicata e specifica. Si integrano solitamente in sistemi più ampi e complessi, che possono coinvolgere sia altri dispositivi elettronici che componenti meccanici in movimento, spesso con il compito di mettere in comunicazione altri dispositivi con il mondo esterno, ricevendo dati e mandando comandi.

Bluewind si occupa sia dello sviluppo di prodotti *software* per aziende nell'ambito industriale, medico e dell'*automotive*, sia di fornire ad altre aziende servizi nell'ambito della sicurezza informatica e della *functional safety*_G (quest'ultima verrà approfondita più nel dettaglio nel [quarto paragrafo](#) di questa sezione).

Dal punto di vista dello sviluppo *software*, gli anni di esperienza nel settore hanno permesso all'azienda di accumulare un bagaglio di conoscenze e competenze che, a oggi, gli permette di occuparsi dell'intero ciclo di vita di un prodotto. L'azienda infatti non si occupa solo dell'implementazione del codice ma è coinvolta in tutti gli stadi del progetto:

- l'analisi dei requisiti
- la scelta e la progettazione dell'*hardware*
- la progettazione del *software*
- la stesura del codice
- lo sviluppo dei *test*
- le verifiche di sicurezza

Occorre tuttavia sottolineare che, in base alle richieste e necessità dei clienti, non necessariamente spetta sempre all'azienda occuparsi di ogni singolo passaggio di questo ciclo di vita.

Un aspetto molto importante del settore in cui opera Bluewind è la presenza, per molti campi di utilizzo dei prodotti *embedded*, di normative e *standard* da rispettare per soddisfare i requisiti di sicurezza di alcuni settori critici. Per questo motivo Bluewind ha raccolto al suo interno una significativa conoscenza sulla *functional safety*, arrivando a offrire servizi di consulenza e di validazione su questo tema ad aziende terze, e riuscendo a fare di questo ambito uno dei suoi principali punti di forza e competitività.

La disciplina della *functional safety* si occupa di garantire che un sistema si comporti in maniera prevedibile e che eventuali errori non portino al rischio di provocare danni.

Un caso pratico che si può prendere in esempio, per dimostrarne l'importanza, è quello di un impianto con vari macchinari ed componenti automatizzati e in movimento, all'interno del quale possono essere presenti degli operatori. In questo caso è fondamentale che il comportamento di tutte le componenti dell'impianto sia prevedibile, e che anche nell'eventualità di un errore *software*, o guasto *hardware*, non si arrivi a una situazione di rischio che possa provocare dei danni a persone o cose.

Questo è quindi un aspetto molto importante, e necessita che i requisiti delle norme e degli *standard* tecnici vengano considerati all'interno del ciclo di vita di un prodotto fin dall'inizio.

Per accertare che durante la progettazione e lo sviluppo siano stati seguiti tali vincoli ci si affida a degli enti certificatori, che fanno una revisione della documentazione relativa al progetto e valutano che poi tutto sia coerente. Recentemente Bluewind ha accolto all'interno del suo *team* alcuni dipendenti che hanno già lavorato in passato come enti certificatori, in modo da poter rafforzare le proprie competenze nella materia.

Ad oggi, Bluewind è presente sul territorio con due sedi: una sede principale, a Castelfranco Veneto (VI), che si occupa delle attività di ricerca, sviluppo e *marketing*, e una sede secondaria a Milano focalizzata sulla *functional safety* e sulla sicurezza informatica in generale.

1.2 L'organizzazione aziendale e il metodo di lavoro

1.2.1 I ruoli

L'azienda è suddivisa in due macro reparti, ognuno dei quali è gestito da un responsabile : il reparto *Sales and Marketing*, incaricato degli aspetti commerciali, pubblicitari, di vendita dei prodotti, e il reparto *Research and Development (R&D)_G* che invece si occupa dell'analisi e sviluppo dei prodotti e della ricerca tecnologica.

Durante la mia attività di tirocinio sono stato inserito all'interno del reparto R&D e ho avuto modo di conoscere i principi alla base del loro metodo di lavoro.

All'interno del reparto, i dipendenti ricoprono diversi tipi di ruoli:

- **Analisti** : si occupano di studiare la realtà in cui si colloca il prodotto, analizzare il problema e stendere quali requisiti è necessario che il prodotto soddisfi, affinché possa adempiere al suo compito. Siccome la realizzazione di un progetto aziendale coinvolge, inevitabilmente, diverse materie, e ognuna di esse richiede specifiche conoscenze, gli analisti si dividono a loro volta in categorie in base allo specifico settore di competenza:
 - **Analisti *hardware*** : siccome Bluewind fornisce e progetta componenti *hardware* su misura, ha nel suo *team* alcuni membri incaricati di capirne le necessità e valutare l'impatto che può avere il malfunzionamento di alcune componenti.
 - **Analisti *software***
 - **Analisti della sicurezza** : si differenziano per la loro conoscenza specifica nei temi della sicurezza informatica, e, nel particolare, delle norme e degli *standard* che ne regolamentano il tema
- **Sviluppatori *software*** : nel caso di Bluewind gli sviluppatori non svolgono solo le attività di stesura del codice, ma assumono anche il ruolo di progettisti, collaborando insieme agli analisti e ai clienti per interpretare al meglio i bisogni di quest'ultimi
- **Specialisti di *functional safety*** : data la rilevanza del tema della *functional safety*, all'interno dell'azienda vi è un gruppo di addetti che si occupa esclusivamente di questa materia
- **Product Owner** : i *Product Owner (PO)_G* sono i responsabili del progetto; si occupano di interfacciarsi con il cliente, gestire la parte di *report* delle attività, supervisionare il *team* di sviluppo, arrivando anche a prendere in carico aspetti burocratici e relativi alla fatturazione

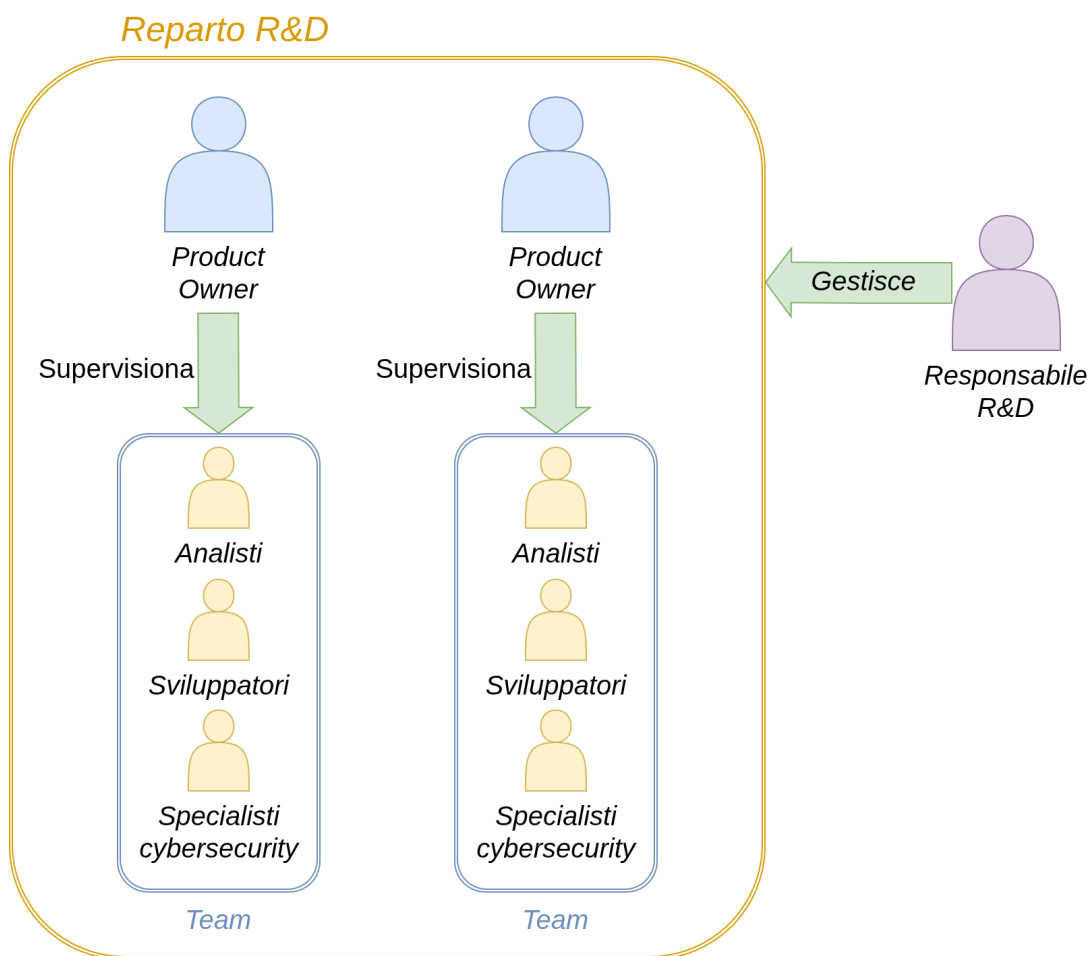


Figura 1.1: Rappresentazione grafica di un esempio di gerarchia dei ruoli all'interno dell'azienda.

Fonte: Elaborazione originale dell'autore

1.2.2 Il metodo *Agile*

Un approccio molto frequentemente adottato per la gestione di progetti e per lo sviluppo *software* è il metodo *agile*, basato su cicli iterativi e incrementali, che privilegia flessibilità, collaborazione, e adattamento al cambiamento rispetto alla pianificazione rigida. All'interno di Bluewind ho avuto modo di osservare come l'adozione del metodo *agile* non sia una prerogativa dello sviluppo *software*, inteso come il processo specifico di progettazione e implementazione di un prodotto, ma che i suoi principi possano integrarsi efficacemente anche all'interno di processi aziendali più generici e ampi.

Da quello che ho potuto cogliere, nell'organizzazione aziendale di Bluewind la metodologia *agile* non si limita a regolare il metodo di avanzamento dei progetti

e delle attività dell' R&D, ma rappresenta anche un importante fondamento del loro modello economico.

Per l'organizzazione dei progetti, l'azienda adotta nello specifico il *framework scrum*_G, un'applicazione del metodo *agile*, dividendo i progetti in *sprint* di una o due settimane, all'interno delle quali sono opportunamente programmate le varie cerimonie previste dal *framework scrum* stesso. I progetti sono assegnati a *team* di circa 3 persone (o più, in base al carico di lavoro), tra sviluppatori e analisti, con alla direzione un *product owner* che volge il ruolo di *scrum master*. Quest'ultimo si occupa quindi di agevolare la corretta applicazione del *framework scrum*.

In base alla necessità è possibile che alcuni dipendenti si occupino nello stesso periodo di più progetti in simultanea, tuttavia è una situazione che si cerca di evitare, spingendo su una suddivisione del lavoro più omogenea tra il personale.

All'inizio della collaborazione, il cliente acquista un determinato numero di *sprint* durante le quali il *team* di Bluewind si andrà a occuparsi del progetto. Agli sviluppatori e agli analisti viene dato l'incarico di definire le attività da svolgere per il proseguimento del progetto, valutando il tempo necessario per ognuna di esse.

In mancanza di una richiesta diversa, esplicita del cliente, una *sprint* si protrae per due settimane, con al suo interno diverse riunioni di sincronizzazione che coinvolgono sia il *team* di sviluppo di Bluewind, sia i rappresentanti della parte cliente. Non di rado accade che nelle riunioni di aggiornamento a confrontarsi sia il personale tecnico delle due parti. Questo permette al cliente di avere una visione molto più nel dettaglio di come procedono i lavori ed, eventualmente, di dare un *feedback* molto più diretto su come si preferisce che vengano gestite certe questioni. Coinvolgere il cliente ai vari stadi del progetto e motivarne la partecipazione, con una frequenza di riunioni che va oltre quella di un approccio *agile* classico, è uno dei principali impegni di Bluewind, che ha saputo dimostrare la sua efficacia.

Esaurito il numero di *sprint* a disposizione, il cliente può decidere se proseguire

con lo sviluppo del progetto acquistandone altre, nel caso desiderasse aggiungere funzionalità o andare ad apportare migliorie. Nella stessa maniera viene gestito anche un eventuale piano di manutenzione e aggiornamento del prodotto.

A questo metodo di gestione del lavoro si legano poi anche altri processi aziendali, che non necessariamente sono coinvolti direttamente nell'evoluzione del prodotto, ma hanno la loro significativa rilevanza sia nella visione del progetto come sua intero, sia a livello di organizzazione aziendale.

1.3 I clienti

Occupandosi di sistemi *embedded*, i domini applicativi dei prodotti e dei servizi di Bluewind spaziano in molteplici mercati. Tra i suoi clienti una buona maggioranza arriva dai settori dell'*automotive*, dell'automazione industriale, dei sistemi di domotica e dell'elettronica di consumo.

Il modo con cui vengono mantenuti i rapporti e le interazioni con questi clienti può cambiare in base al loro settore di appartenenza. Un chiaro esempio è il mondo dell'*automotive*: all'interno dell'industria automobilistica esiste una gerarchia tra le aziende, che colloca ogni azienda in un livello specifico in base al suo ruolo nella produzione del prodotto. Al vertice della gerarchia si collocano le case automobilistiche, chiamate *Original Equipment Manufacturer (OEM)_G*, le quali decidono di delegare la produzione di alcune componenti ad altre aziende. L'azienda che riceve l'incarico dall'OEM prende il nome di azienda *tier 1*. Se successivamente essa decide di delegare, a sua volta, parte del lavoro ad altre aziende quest'ultime prenderanno il nome di *tier 2*. Lo stesso ragionamento si applica per ogni livello di delega che si aggiunge. Generalmente Bluewind si colloca tra il *tier 1* e il *tier 2*.

Nell'ambito industriale, invece, non avviene questo tipo di suddivisione, e l'azienda lavora direttamente con il produttore del sistema completo. Questo cambia il tipo di relazione che Bluewind ha con l'azienda committente e il progetto;

nel caso di lavori svolti per delle OEM, Bluewind non ha modo di vedere come si integra il suo operato all'interno del progetto principale, in quanto la sua visione rimane limitata al singolo elemento che le è stato affidato. Nelle altre casistiche ha invece modo o di essere coinvolta direttamente con tutto il ciclo di vita del prodotto, qualora le venga commissionato l'intero processo, o quantomeno di poter cogliere il funzionamento del sistema nel suo intero.

Tipologie di interazioni con i clienti

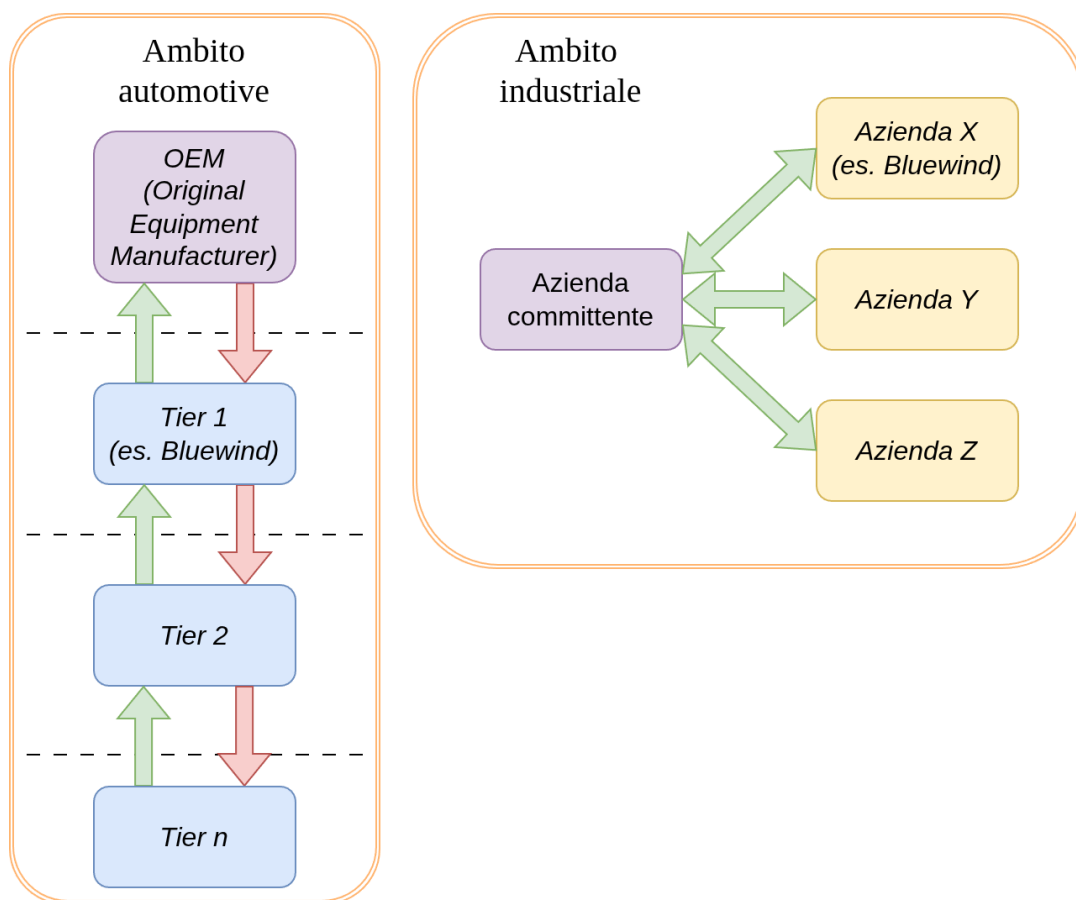


Figura 1.2: Schema che rappresenta la differenza di interazioni tra i clienti in ambito *automotive* e quelli in ambito industriale generale

Fonte: Elaborazione originale dell'autore

1.4 Lo *stack* tecnologico aziendale

1.4.1 Visione generale sull'adozione delle tecnologie

Gli anni di attività hanno portato Bluewind a prendere confidenza con diverse tecnologie e strumenti, che spaziano in vari casi di utilizzo. La loro adozione tuttavia è fortemente influenzata dalle richieste del cliente. Difatti, Bluewind tende a non avere uno *standard* fisso su quali strumenti utilizzare, e si rifà molto a quello che viene utilizzato dall'azienda cliente. Questo avviene perché lavorando a stretto contatto con i propri clienti, e i rispettivi *team* di sviluppo, spesso su progetti già avviati, è necessario per Bluewind allinearsi con il metodo di lavoro del cliente. Spesso tale scelta è motivata anche dal fatto che il prodotto finale comprende anche il codice sorgente, il quale rimane al cliente che potrebbe volerci lavorare ulteriormente in futuro.

Nonostante questa dinamica, vi sono delle tecnologie che a cui ricorrono con maggiore frequenza o alle quali, in mancanza di specifiche richieste, preferiscono affidarsi.

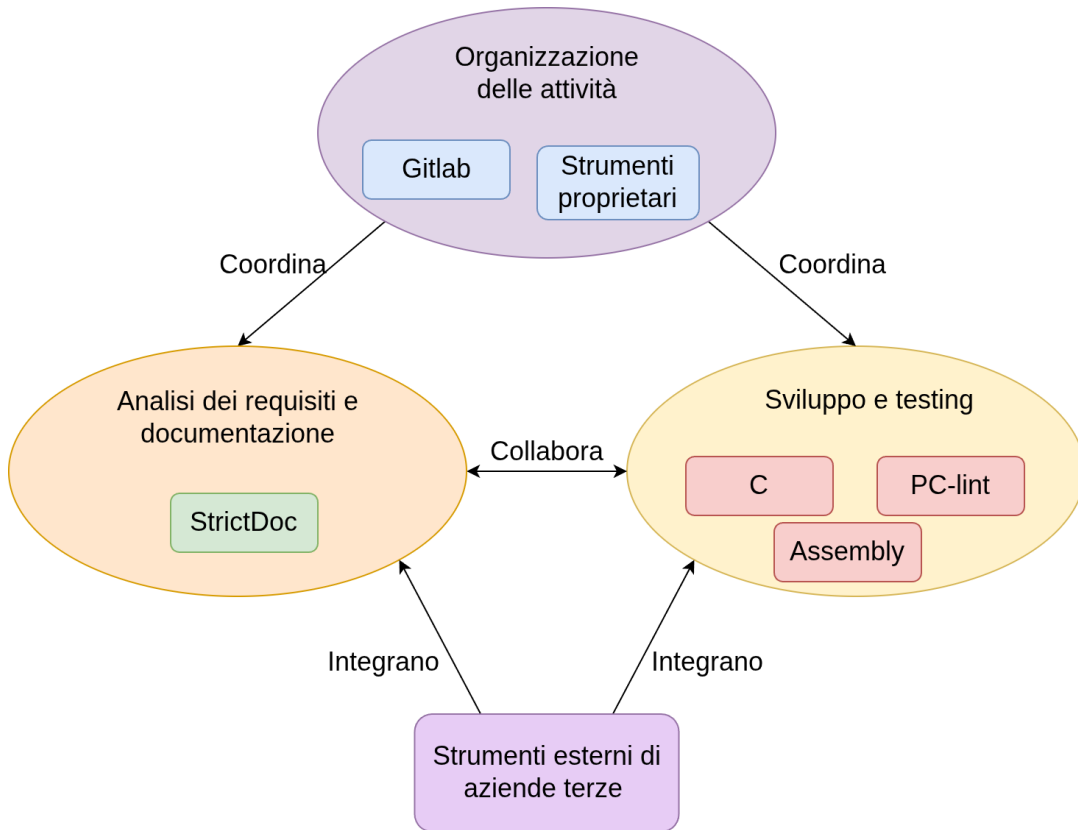


Figura 1.3: Schema che rappresenta l'interazione delle tecnologie all'interno dei processi aziendali.

Fonte: Elaborazione originale dell'autore

1.4.2 Gitlab

GitLab è una piattaforma che integra in un unico strumento la gestione del codice sorgente, il controllo di versione basato su Git, e la gestione dei progetti. Permette inoltre di adottare l'integrazione e distribuzione continua, *CI/CD*, ossia la possibilità di automatizzare il rilascio del *software* e l'avvio di una sequenza di *test*. Come ci si può aspettare, Bluewind adotta questo strumento integrandolo all'interno dei propri *server*, e rendendolo un elemento centrale all'interno della sua organizzazione aziendale. Questo strumento si integra molto bene con il metodo *agile* adottato dall'azienda, permettendo di organizzare, suddividere e tenere traccia delle attività in maniera più organizzata e meticolosa.

1.4.3 Servizi e strumenti proprietari

Oltre ad affidarsi a tecnologie e strumenti esterni, Bluewind ha una sua *suite* di strumenti proprietari per la gestione dei suoi processi interni. Lo scopo principale è offrire servizi che agevolino alcuni compiti : l'organizzazione tra il personale, controllare la disponibilità dei vari dipendenti, organizzare incontri, gestire lo *smart working*, oppure, andando più sul tecnico, permettere di accedere da remoto a dispositivi *hardware* condivisi tra più utenti e utili allo sviluppo. Quindi, seppur questi servizi non siano necessariamente coinvolti in maniera diretta nell'evoluzione del progetto, la loro presenza è importante per facilitare il coordinamento dei processi principali, e per garantire che il loro svolgimento sia più fluido.

1.4.4 StrictDoc

StrictoDoc è uno strumento utilizzato per la scrittura di documentazione, gratuito e *open source* (ovvero il cui codice è pubblicamente disponibile e modificabile liberamente dall'utente). Viene adottato specialmente per l'analisi dei requisiti in quanto permette di fare più facilmente riferimento tra i requisiti e al codice, di creare la matrice di tracciabilità e di esportare la documentazione in altri formati, anche quelli usati da *tools* proprietari. La possibilità di associare documentazione direttamente al codice di interesse lo rende uno strumento ampiamente adottato anche dai programmatori, sia per indicare dove vengono implementati i requisiti, sia per aggiungere chiarezza sul lavoro svolto laddove necessario. È uno strumento su cui Bluewind sta puntando molto per via della sua semplicità di utilizzo e della possibilità, in quanto *open source*, di aggiungere funzionalità *custom* in base alle proprie esigenze.

1.4.5 Il linguaggio C

Il linguaggio C rimane il linguaggio dominante nel settore *embedded*, in quanto offre un controllo quasi diretto sull'*hardware* (registri, memoria, *interrupt*) mantenendo comunque un livello di astrazione superiore all'Assembly. Su mi-

crocontrollori con risorse limitate (specialmente in termini di memoria, si parla anche pochi *kilobyte* di *RAM*) non c'è modo di adottare linguaggi più moderni con funzionalità più complesse come il *garbage collector* o l'*overhead*, occorre invece qualcosa che offra prevedibilità nei tempi di esecuzione e controllo esplicito sull'allocazione. Su questo tema il linguaggio C rimane ancora oggi il punto di riferimento per il settore. Va detto però che questo controllo è anche notoriamente il suo punto debole: niente *memory safety* nativa, puntatori pericolosi, comportamenti *undefined* sono rischi che negli ambiti *safety-critical* (come quello *automotive*, o medico) vanno trattati con assoluto riguardo.

Nel capitolo successivo parlerò più a fondo delle regole *MISRA_C* che rappresentano un importante *standard* per limitare gli errori più comuni.

1.4.6 Il linguaggio Assembly

Seppur non accada con grande frequenza, il C a volte non è sufficiente, e si rende necessaria l'adozione del linguaggio Assembly per raggiungere il livello di controllo richiesto. L'Assembly resta il linguaggio più basso disponibile nell'*embedded*, avendo una corrispondenza vicina all'uno a uno con le istruzioni macchina dell'architettura. Quindi viene utilizzato quando serve controllo assoluto sull'*hardware*, accesso a istruzioni specifiche non esposte dal C o un'ottimizzazione estrema di alcune funzioni critiche.

1.4.7 PC-lint

PC-lint è uno strumento di analisi statica per C e C++ utilizzato in ambienti *safety-critical* per scoprire *bug*, vulnerabilità e verificare il rispetto delle regole imposte dagli *standard*, tra cui anche le regole MISRA.

È uno strumento *CLIC* e può essere integrato all'interno in un processo automatizzato di *test* e *deployment*.

1.4.8 Strumenti proprietari di aziende *partner* come Infineon e ST

Nell'ambito *embedded* è spesso richiesto l'utilizzo di componenti *hardware* costruite su misura per il loro caso di applicazione. Ne consegue che, di progetto in progetto, ogni dispositivo per cui andrà sviluppato il *software* sia leggermente diverso, per le componenti (connettori, microprocessori, adattatori ecc.) di cui è composto. Per questo motivo, le aziende fornitrici di *hardware* come Infineon Technologies e STMicroelectronics (da notare che questi sono i *brand* che io ho avuto modo di conoscere nella mia, relativamente breve, esperienza, tuttavia lo stesso ragionamento si applica anche ad altri *partner* aziendali) mettono a disposizione delle schede che racchiudono un insieme predefinito di componenti che possono essere presenti nei dispositivi finali. In questo modo le aziende produttrici di *software* possono utilizzare le singole componenti di loro interesse, senza avere ancora a disposizione i dispositivi *hardware* definitivi. Questo semplifica lo sviluppo, rendendo più semplice per i programmatori ottenere un *feedback* sul funzionamento del codice.

Oltre alle schede, le aziende produttrici mettono a disposizione un gran catalogo di risorse per agevolarne l'utilizzo e la scrittura del codice, quali documentazione dettagliata sui prodotti, ambienti di sviluppo (*IDE_G*) specifici per il tipo di scheda e altri *tools* di monitoraggio e *debugging*.

1.5 Propensione all'innovazione

In un periodo come quello attuale, in cui l'avanzamento tecnologico lascia poco spazio all'indecisione, avere una chiara idea e un chiaro metodo di approccio all'innovazione è fondamentale per potersi posizionare correttamente all'interno del mercato e per poter impiegare al meglio tempo e risorse. Per questo motivo all'interno di Bluewind è presente un impegno molto sentito verso il tema dell'innovazione che coinvolge tutto il personale.

Il loro specifico mercato di riferimento tuttavia non li aiuta: lavorare in settori dove la sicurezza e la prevedibilità dei sistemi sono aspetti profondamente critici, porta le aziende a dover avanzare con cautela nell'esplorare nuove opportunità. Un ostacolo concreto sono i vincoli posti dai requisiti obbligatori a cui le aziende devono sottostare.

Un chiaro esempio di queste limitazioni è l'adozione dell'IA: l'utilizzo dei algoritmi di *machine learning*_G, atti ad apprendere dei *pattern* dai dati con lo scopo di prevederne il progresso, è ancora un argomento non del tutto esplorato vista la mancanza di regolamentazione a riguardo, ma soprattutto perché aggiunge una variabile imprevedibile all'interno di un contesto che spesso non se la può permettere.

Nonostante ciò ci sono stati dei passi avanti da parte dell'azienda anche su questo tema, e persiste il desiderio di approfondire anche questa tematica.

In generale, Bluewind segue un approccio molto attivo sul tema dell'innovazione, difatti ho notato in molti dipendenti la predisposizione ad accogliere le novità e a interessarsi fin da subito a come trarne vantaggio. Oltre a ciò l'azienda intraprende costantemente iniziative volte al miglioramento dei propri processi interni, sia tramite collaboratori esterni, sia con ricerche svolte dai dipendenti stessi.

Una parte importante in questo piano sono i tirocini studenteschi, i quali permettono a Bluewind di esplorare nuove tematiche e soluzioni per le quali altrimenti non ci sarebbero il tempo o le risorse da dedicare.

Capitolo 2

L'origine e gli scopi del tirocinio

In questo capitolo spiego il ruolo dei tirocini all'interno di Bluewind, contestualizzandoli in relazione alla loro visione di innovazione. Vado poi a descrivere l'attività di tirocinio che mi è stata proposta, quali motivazioni hanno spinto Bluewind a ideare quest'iniziativa e con quali obiettivi. Infine, do le mie, personali, motivazioni che mi hanno portato a scegliere questa proposta di stage.

2.1 Gli *stage* all'interno del piano aziendale

Come già anticipato nella sezione [1.5](#), la continua necessità e ambizione di Bluewind di rimanere al passo con il progresso tecnologico, porta l'azienda a riservare tempo e personale per inseguire tale obiettivo. Non è però scontato riuscire ad avere sempre a disposizione i mezzi per fare ciò, soprattutto visto l'avanzamento sempre più rapido del settore informatico.

Per questo motivo l'azienda cerca di sfruttare appieno l'opportunità rappresentata dai tirocini, lavorando a stretto contatto con le università. I tirocini universitari permettono infatti a Bluewind di esplorare con più facilità le tematiche di suo interesse. Vengono raccolti nel tempo spunti e idee su tecnologie, strumenti e progetti che potrebbero essere di impatto per la crescita dell'azienda, e partendo da essi si vanno a creare delle proposte di tirocinio con lo scopo valutarne l'adozione. Le proposte di tirocinio possono dunque avere sia come obbiettivo la ricerca dei suddetti temi, sia la creazione di un *Proof of Concept*

$(PoC)_G$, un prodotto dimostrativo che serve a valutare la fattibilità e l'utilità di un progetto prima che ne si cominci la realizzazione vera e propria. A partire dai risultati ottenuti, l'azienda può poi decidere se è opportuno proseguire a esplorare o meno la tecnologia in questione, avendo anche, nel caso del PoC, una base concreta dalla quale poter partire per lo sviluppo di un progetto.

Gli *stage* rappresentano inoltre un'opportunità per l'azienda nel merito delle assunzioni. I tirocini permettono infatti all'azienda di seguire da vicino ragazzi giovani e prossimi alla laurea, valutando con concretezza la possibilità di inserirli all'interno della propria attività. Questa modalità di *scouting*, che ha, negli anni, portato all'assunzione di diverse persone, ha oltretutto il vantaggio di agevolare l'ambientamento da parte dei nuovi dipendenti, in quanto essi hanno già avuto modo di conoscere quella realtà.

2.2 La spinta normativa

2.2.1 Cyber Resilience Act

Come anticipato nel capitolo precedente, Bluewind, per poter operare nel suo settore, necessita di allinearsi con una serie di direttive europee e *standard* che regolano la produzione e la messa in commercio dei prodotti. Occorre dunque, per poter comprendere e interpretare al meglio certe dinamiche e decisioni del contesto aziendale di Bluewind, quantomeno introdurre tale tema.

Una delle prime normative su cui occorre riporre l'attenzione è il *Cyber Resilience Act (CRA)* $_G$, una normativa dell'Unione Europea che copre tutti i dispositivi con elementi digitali o che siano connessi con altri dispositivi e/o servizi, insieme nel quale ricadono dunque anche l'elettronica di consumo e la domotica, di cui si occupa Bluewind. Il CRA introduce dei requisiti di sicurezza informatica che i costruttori devono dimostrare di rispettare in tutte le fasi del ciclo di vita dei loro prodotti, dalla pianificazione, allo sviluppo, fino alla messa in commercio e manutenzione. Questa normativa obbliga i costruttori a gestire, al loro in-

sorgere, le vulnerabilità del proprio prodotto durante tutto il suo ciclo di vita. In alcuni casi di particolare rilevanza per la sicurezza, può essere addirittura necessario che un prodotto passi una revisione aggiuntiva, da parte di enti terzi, per poter essere introdotto all'interno del mercato europeo.

2.2.2 Regolamenti europei No. 155 e No. 156

Andando poi ad approfondire più nello specifico le regolamentazioni per il settore dell'*automotive*, troviamo i regolamenti europei R155 e R156, i quali contengono requisiti obbligatori per l'omologazione dei veicoli in Europa. Il R155 richiede che i costruttori adottino un metodo di lavoro, ben definito (e certificato), che preveda dei processi focalizzati sull'identificazione e sulla risoluzione dei rischi informatici, attraverso tutto il ciclo di vita del veicolo. Il regolamento espone inoltre i principali rischi, che possono esserci, e i loro metodi di mitigazione. Il regolamento R156 definisce invece i requisiti necessari durante l'aggiornamento delle componenti *software* all'interno dei veicoli, in modo che questi siano sicuri e tracciabili.

Questi due regolamenti specificano il **cosa** vada implementato all'interno delle soluzioni *software*. Il **come** questo venga fatto trova una risposta all'interno degli *standard* ISO-21434 e ISO-24089 (rispettivamente per R155 e R156).

2.2.3 *Standard* ETSI EN 303645

Delle linee guida esistono anche per quanto riguarda la più generale elettronica di consumo. A fare da riferimento per tale settore è lo *standard* ETSI EN 303645; di adozione inizialmente europea, è finito per diventare il fondamento per la sicurezza dei dispositivi *IoT_G*, anche a livello globale.

A differenza dei regolamenti visti in precedenza, questo *standard* si concentra sul risultato delle misure intraprese e non sulla loro implementazione. Questo avviene con l'intento di dare maggiore flessibilità alle organizzazioni, in modo che esse possano studiare, e migliorare, delle soluzioni che siano il più appropriate possibile per i loro prodotti.

Inoltre, l'intento non è assicurare la protezione da tutti i rischi possibili, ma di tracciare una linea di partenza, un livello base, di sicurezza che possa evitare le vulnerabilità più comuni. Si prevede perciò che questo *standard* venga completato con l'adozione di altri *standard* aggiuntivi, che prevedano l'adozione di misure più specifiche, pertinenti a quelli che sono gli obiettivi finali del progetto.

2.2.4 Regole MISRA

Si può quindi facilmente capire che affinché un prodotto, sia *hardware* che *software*, possa essere inserito all'interno del mercato occorre svolgere un esteso lavoro preliminare su esso, in modo da garantire il rispetto di tutti i requisiti.

A questi requisiti se ne aggiungono anche altri, più comunemente definiti dalle aziende, che riguardano la qualità del codice attesa. Uno di questi è il rispetto delle regole MISRA, che sono uno *standard de facto* nello sviluppo di codice in C in quegli ambiti in cui la sicurezza e qualità del codice hanno un ruolo centrale (ambiti in cui si colloca Bluewind).

Ampliamente usate in ambito *embedded*, le regole MISRA C definiscono delle linee guida per lo sviluppo di codice in C, con l'obiettivo di indirizzare lo sviluppo verso la creazione di codice più mantenibile e libero da potenziali rischi per la sicurezza. Il linguaggio C concede una significativa libertà di azione agli sviluppatori, la quale spesso e volentieri è la causa di *bug* che possono sfociare in comportamenti indesiderati del programma, fino anche a compromettere lo stato del sistema. Specialmente in applicazioni che interagiscono con componenti esterne, la presenza di tali rischi non è accettabile.

Le regole MISRA limitano quindi il linguaggio C, restringendone in un certo senso le funzionalità. L'obiettivo è eliminare la possibilità che si verifichino comportamenti inattesi, e di promuovere la scrittura di codice più facilmente mantenibile.

2.3 Il progetto e la sua finalità

La crescente quantità di vincoli che i produttori sono tenuti a rispettare ha dato origine, all'interno di Bluewind, all'esigenza di formulare delle strategie che contengano questo incremento di complessità. Quest'esigenza si estende sia sulla questione burocratica e amministrativa dei progetti, sia sullo sviluppo. Viene infatti chiesto alle aziende di provare la corretta implementazione delle misure di sicurezza richieste, attraverso *test* oggettivi e ripetibili.

Per cercare delle soluzioni che facilitino questo compito, Bluewind ha ideato l'attività di tirocinio del quale mi sono andato a occupare.

Lo scopo di questo tirocinio è stato progettare e prototipare un *framework* di *test* di sicurezza informatica, incentrato nello specifico sulla verifica di dispositivi *embedded*. Il progetto prevedeva l'implementazione concreta di *test* di sicurezza che rappresentassero dei scenari di utilizzo improprio, e che quindi potessero fornire informazioni sulla robustezza del prodotto.

Oltre a ciò, due punti indicati come di particolare interesse da quest'attività sono stati: l'esecuzione su *hardware* reale dei *test*, e la possibilità di avviare tale esecuzione da remoto.

2.4 Gli obiettivi del progetto

2.4.1 Obiettivi obbligatori

Prima che iniziassi nel pratico l'esperienza di tirocinio, io e il mio *tutor* aziendale abbiamo definito chiaramente quali fossero gli obiettivi per il progetto nel quale mi sarei cimentato.

Inanzi tutto abbiamo specificato quelli che sarebbero stati gli obiettivi minimi obbligatori. Al completamento di questi sarebbe risultato un prodotto molto basilare, il cui sviluppo però mi avrebbe portato a prendere dimestichezza con

le tecnologie, e soprattutto ad apprendere le basi teoriche necessarie poi per il proseguimento del progetto. Seguono i suddetti obiettivi:

- **Setup di un canale di comunicazione CAN_G :** CAN è un protocollo di comunicazione utilizzato nei veicoli per mettere in comunicazione tra loro le varie centraline.
- **Definizione di una struttura che implementi un *test runner*:** mi sono stati consegnati, all'inizio del progetto, degli *script* in linguaggio Python contenenti delle funzioni che effettuavano dei *test* di sicurezza. Questo obiettivo prevedeva la creazione di una soluzione *software* che permettesse di lanciare questi *test* tramite il canale CAN precedentemente creato. Successivamente questi stessi *script* sono stati la base di partenza da cui ho progettato e sviluppato le classi di *test* (verranno affrontate nel dettaglio nella sezione 3.4.3).
- **Sviluppo di un'API $_G$ per l'esecuzione di *test* da remoto:** un'API è un meccanismo che mette a disposizione dei servizi per far interagire due o più *software*. In questo punto del progetto l'API aveva lo scopo di agevolare l'interazione con l'operatore; la sua implementazione è poi progredita arrivando a essere una componente chiave del progetto (ne parlo più nel dettaglio nella sezione 3.4.6).

2.4.2 Obiettivi desiderabili

Abbiamo poi previsto una serie di obiettivi desiderabili, la cui realizzazione ha occupato la maggior parte delle ore previste dall'esperienza.

Completati questi obiettivi, ne sarebbero risultati una libreria che implementasse l'esecuzione dei *test* in tutte le sue parti, e un'applicazione dimostrativa che raggruppasse il tutto in un processo unico.

Seguono i suddetti obiettivi:

- **Refactoring dei *test* esistenti:** questo obiettivo prevedeva il riutilizzo del materiale che mi è stato fornito per progettare e implementare i *test* in modo che potessero essere riutilizzati all'interno di un sistema più grande.

- **Sviluppo di un sistema modulare di *test*:** questo obiettivo prevedeva lo sviluppo di un sistema che gestisse il *setup*, l'esecuzione e la conservazione dei *test*. Per fare ciò si è previsto già dall'inizio di utilizzare componenti distinte (e sostituibili) che si occupassero ognuna di un singolo compito specifico.
- **Sviluppo di un'applicazione dimostrativa:** questo obiettivo prevedeva il creare un'applicazione che coordinasse l'insieme di componenti citato nel punto precedente per gestire l'esecuzione dei *test* dall'inizio alla fine. Ho poi deciso, insieme al *tutor*, che l'applicazione sarebbe stata rappresentata dall'API stessa.

2.4.3 Obiettivi facoltativi

Sono stati previsti anche degli obiettivi facoltativi a cui dedicarsi qualora, una volta soddisfatti tutti gli obiettivi obbligatori e desiderati, fosse rimasto del tempo a disposizione. Tali obiettivi sono lo sviluppo di una interfaccia grafica da cui lanciare i *test*, e lo sviluppo di ulteriori *test* aggiuntivi che si basino su altri protocolli di comunicazione, diversi dal CAN.

Questi obiettivi hanno lo scopo di mostrare se il prodotto ottenuto fino a quel momento sia o meno facilmente espandibile. Infatti, l'impegno e il tempo necessari per sviluppare queste funzionalità aggiuntive sarebbero stati il metodo di giudizio per verificare che tutto quanto il progetto sia stato svolto con una qualità apprezzabile.

2.5 Le ambizioni personali

2.5.1 La scelta del tirocinio

Nel valutare le proposte di tirocinio che ci sono state presentate, ho deciso di concentrarmi su pochi criteri ma ai quali ho inteso attenermi fermamente.

Il mio intento principale era andare a lavorare a un progetto, e in una realtà, che avesse una forte affinità con l'ambito *embedded*. Nel corso della mia carriera

di studi ho avuto modo di mettere mano a diversi linguaggi di programmazione ad alto livello, e di conseguenza la mia esperienza si è concentrata prevalentemente su progetti orientati all'utente finale, quali applicazioni e siti *web*. Molto raramente mi sono invece interfacciato all'ambito della programmazione a basso livello, e in quelle poche occasioni non sono comunque mai riuscito a lavorare su applicazioni che non fossero puramente teoriche. Causa di ciò è in parte anche la scarsa offerta di opportunità, da parte dell'università, per approfondire questo settore, il quale, nonostante tutto, mi ha sempre destato un certo interesse. Ho deciso quindi di approfittare di questa attività di tirocinio per farmi una prima esperienza concreta in questo settore, ponendo l'attenzione sulle offerte che più mi avrebbero permesso di lavorare su dispositivi e protocolli *embedded*.

Un'altra caratteristica che ho tenuto in considerazione è stata la dimensione dell'azienda. Nelle mie esperienze di lavoro precedenti ho lavorato in aziende con davvero pochi dipendenti, e avevo quindi il desiderio provare a inserirmi in una realtà più grande. Questo mio desiderio è spinto dalla volontà di conoscere ambienti aziendali più complessi e vedere come si svolgono processi che comprendano un numero considerevole di persone. Inoltre, mi era naturale pensare che un'azienda di grandi dimensioni si occupasse anche di progetti più complessi e con un carico di lavoro maggiore; era di mio interesse capire come si lavora a progetti del genere, che sicuramente necessitano di un metodo di lavoro e un'organizzazione che difficilmente avrei trovato in aziende con pochi dipendenti.

2.5.2 Gli obiettivi personali

Essendo questa esperienza, sotto un certo punto di vista, la chiusura del ciclo di studi, volevo mostrare di aver acquisito un metodo di lavoro solido sia nell'acquisire conoscenze, sia nello svolgere il lavoro in maniera corretta e con un risultato apprezzabile.

Per dare dimostrazione di ciò, mi sono posto degli obiettivi che mettessero alla prova queste capacità.

- **Raggiungere l'autonomia nell'utilizzo delle tecnologie:** mostrare che sono in grado di studiare, apprendere e sfruttare coscientemente le proprietà delle tecnologie con cui lavoro. Questo obiettivo comporta l'avere una completa autonomia nell'utilizzo degli strumenti e tecnologie che occorrono per svolgere il lavoro richiestomi, e pone particolare attenzione sulla mia capacità di trovare autonomamente le informazioni di cui necessito.

Ho deciso di considerare questo obiettivo soddisfatto se nelle ultime settimane di tirocinio riuscirò ad adempiere a tutti i compiti che mi sono posto senza la necessità di chiedere assistenza al mio *tutor*, anche nel caso mi trovassi a dover utilizzare funzionalità degli strumenti delle quali, fino a quel momento, non avevo ancora usufruito.

- **Risoluzione consapevole dei problemi:** questo obiettivo si concentra sul mostrare che sono in grado di analizzare i problemi che mi vengono proposti, o che incontro durante il progetto, e trovare a essi una soluzione adatta e coerente con il resto del progetto.

Questo mio obiettivo si lega molto agli obiettivi facoltativi del progetto, esposti nella sezione [2.4.3](#). Ritengo infatti che mostrare l'estendibilità del progetto, senza sforare i tempi inizialmente definiti, provi che il progetto e le sue parti siano stati pensati e sviluppati tenendo bene conto del loro insieme. Riterrò quindi questo obiettivo soddisfatto se riuscirò a soddisfare tutti gli obiettivi facoltativi nel tempo previsto.

- **Valorizzare il lavoro svolto** Questo punto prevede la messa in pratica delle conoscenze apprese durante il corso di Ingegneria del *Software*, allo scopo di ottenere il massimo valore possibile dal tempo investito sul lavoro. Ho deciso di utilizzare come metodo di giudizio di questo punto la quantità di obiettivi raggiunti per il progetto.

Capitolo 3

Lo sviluppo del progetto

In questo capitolo parlo del progetto in se e per se, analizzando il problema che mi è stato posto e come ho deciso di risolverlo. Vado poi a fare un resoconto finale sul prodotto ottenuto e ne valuto la qualità.

3.1 Il metodo di lavoro

All'inizio della mia collaborazione con l'azienda, ho stilato, insieme al mio *tutor* aziendale, un piano di lavoro per lo *stage* che sarei andato a svolgere. Tale documento contiene una suddivisione delle attività previste per ognuna delle otto settimane della mia permanenza in azienda.

Abbiamo deciso quindi, insieme al *tutor* aziendale, che avrei seguito fedelmente questa suddivisione, organizzandomi il lavoro in modo da riuscire, ogni settimana, a completare ciò che veniva richiesto. Le attività sono organizzate in modo da arrivare a soddisfare tutti gli obiettivi entro la fine dello *stage*, tenendo conto che qualora un'attività necessiti di più tempo di quello previsto si andrà a continuarla nella settimana successiva fino a che non è finita. Abbiamo ordinato il lavoro in modo che gli obiettivi vengano soddisfatti in base alla loro importanza, partendo quindi prima da quelli obbligatori, poi quelli desiderabili e infine i facoltativi.

Settimana	Durata in ore	Descrizione attività
Prima	40	Scelta delle tecnologie, setup dell'ambiente di sviluppo e definizione dei requisiti minimi
Seconda	40	Setup del canale di comunicazione (CAN) ed esecuzione manuale di test
Terza	40	Definizione di una struttura per descrivere i test e implementazione di un test runner
Quarta	40	Sviluppo di API per l'esecuzione di test da remoto
Quinta	40	Refactoring test esistenti
Sesta	40	Refactoring e modularizzazione del sistema di test
Settima	40	PoC dell'estendibilità del sistema tramite setup di un altro canale di comunicazione (es. UART)
Ottava	20	Documentazione e sviluppo di un'applicazione dimostrativa
300 ore totali		

Figura 3.1: Estratto del documento "Piano di lavoro" contenente la suddivisione delle attività per ogni settimana prevista dal tirocinio.

Fonte: Elaborazione originale dell'autore

All'inizio di ogni attività suddividevo il lavoro in più compiti atomici che avrei poi svolto durante quel periodo. Nel particolare ho utilizzato una *board* di Gitlab, all'interno della quale segnavo, e tenevo nota, di tutte le attività pianificate e le dipendenze che avevano tra di loro. Questo mi permetteva di avere una chiara visione dell'andamento e dell'ordine in cui svolgere le attività, senza incorrere in dipendenze inaspettate che interferissero con un fluido andamento del lavoro.

Alla fine di ogni settimana era previsto un resoconto (di persona) durante il quale mostravo al mio *tutor* il risultato di quella settimana di lavoro, ricevevo un *feedback* e gli spiegavo quali attività prevedevo di svolgere per la settimana successiva. Oltre a questo resoconto settimanale, si aggiungevano le interazioni

quotidiane con il mio *tutor* nelle quali gli facevo note le difficoltà che incontravo e i progressi fatti. Questa stretta interazione ci permetteva sia di rimanere sempre allineati sullo stato del progetto, sia di affrontare eventuali problematiche sul nascere.

3.2 Le tecnologie utilizzate

Per la realizzazione di questo progetto ho usufruito di un notevole numero di tecnologie, dai linguaggi di programmazione, ai protocolli di comunicazione, fino ad arrivare a IDE e *hardware* proprietari, specifici per lo sviluppo *embedded*.

3.2.1 Linguaggi e *framework*

- **Linguaggio Python** : linguaggio di programmazione ad alto livello, noto per la sua versatilità in numerosi ambiti e casi d'uso. La scelta di utilizzare Python come linguaggio principale per questo progetto risiede nella presenza di diverse librerie, già collaudate, per l'implementazione dei vari protocolli di comunicazione che vengono usati nel progetto.
- **Linguaggio C** : linguaggio di programmazione a basso livello, usato comunemente nello sviluppo *embedded* (già introdotto nel punto [1.4.5](#)). Il linguaggio C è stato necessario per sviluppare il *firmware* di una delle schede in modo che potesse rispondere alle richieste inviate dal programma principale.
- **FastAPI** : *framework* per la realizzazione di API in Python. Conosciuto per le sue prestazioni apprezzabili e per la semplicità di adozione. Ne consegue che l'ho scelto per l'implementazione dell'API all'interno del progetto. Comodo soprattutto perché riconosce automaticamente gli *endpoint* e, senza il bisogno di un intervento diretto del programmatore, mette a disposizione un'interfaccia per inviare richieste.
- **Svelte** : *framework* gratuito e *open source* per lo sviluppo di applicazioni *web* per il *frontend*. Comprende un compilatore che trasforma il codice Svelte in codice JavaScript, in modo che possa essere eseguito sui *browser*.

Ho deciso di scegliere questo *framework* per sviluppare l'interfaccia grafica più per una questione di semplicità. La componente *frontend* non è mai stata considerata centrale all'interno del progetto, avendo uno scopo più dimostrativo che funzionale. Ho deciso quindi di premiare un *framework* che avesse come punto di forza la facilità di adozione.

3.2.2 Protocolli

Come verrà spiegato più avanti in questa sezione (sottosezione 3.2.3), sono state utilizzate due schede diverse come *target* per i *test* eseguiti. Questo perché non tutti gli attacchi potevano venire effettuati attraverso lo stesso canale di comunicazione. Si è reso necessario dunque l'utilizzo di due schede separate, che supportassero ognuna un diverso tipo di comunicazione.

Il primo tipo di comunicazione, nonché quello più complesso, era costituito dal seguente insieme di protocolli:

- **CAN *Bus*** : Il CAN *Bus*, o ISO-11898, è un protocollo di comunicazione seriale, sviluppato per consentire lo scambio di informazioni tra unità di controllo elettroniche all'interno di un sistema complesso. È largamente usato nel settore *automotive* per mettere in comunicazione le varie componenti elettroniche che sono presenti all'interno di un veicolo. Per questo progetto, il CAN *Bus* sarà il protocollo alla base della comunicazione con una delle due schede usate come bersagli dei *test*.
- ***IsoTp_G*** : ISO-15765-2, è un protocollo di trasporto che permette di inviare messaggi tramite comunicazione CAN eliminando il vincolo iniziale di avere al massimo messaggi di otto *byte*. Il messaggio iniziale viene diviso in più pezzi, ai quali vengono aggiunti dei metadati che permettano poi di ricostruire il messaggio una volta ricevuto.
- ***Unified Diagnostic Services (UDS)_G*** : è un protocollo di comunicazione utilizzato nel settore *automotive* per comunicare con le centraline (ECU) presenti sui veicoli. Attraverso dei codici esadecimali è possibile

intervenire sulla centralina, richiedendo informazioni, lanciando comandi o aggiornandone il *firmware*. Una grossa parte degli attacchi sviluppati saranno rivolti proprio a verificare quali risorse e funzionalità sono accessibili tramite la comunicazione instaurata (parlerò più a fondo dei tipi di attacchi nella sezione 3.3 - Analisi).

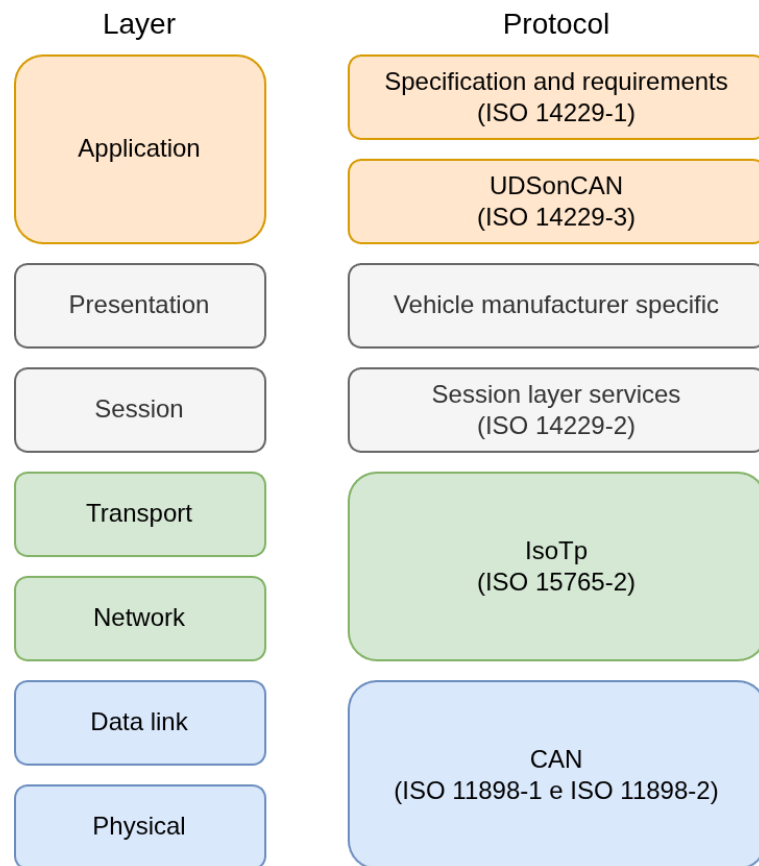


Figura 3.2: Schema che rappresenta i protocolli utilizzati per la comunicazione CAN all'interno della pila ISO-OSI.

Fonte: Elaborazione originale dell'autore, basata sullo schema fornito dal sito [CSS Electronic](#)

La seconda comunicazione è invece effettuata tramite protocollo *Modbus*, un protocollo di comunicazione molto comune nel settore industriale che permette di andare a leggere e scrivere le risorse dei dispositivi (chiamati *slave*) all'interno di un sistema. È un protocollo ampiamente adottato soprattutto per via della sua semplice implementazione e manutenzione. Può tuttavia nascondere falle di sicurezza, lasciando libero accesso a risorse sensibili, che non dovrebbero poter essere accessibili da chiunque. Questa sua caratteristica l'ha reso un bersaglio

di interesse per questo progetto.

A differenza della prima comunicazione, la quale utilizza un'interfaccia fisica supportata dal CAN per inviare i messaggi, questo canale trasmetterà i messaggi tramite una periferica *UART_G*, la quale mette a disposizione una comunicazione seriale per l'invio e il ricevimento dei dati.

3.2.3 Schede *hardware*

- Infineon Aurix TC375 Lite Kit
- ST Nucleo-H533RE

3.2.4 Strumenti proprietari esterni

- STM32cubeIDE
- STM32cubeMx

3.3 Analisi

3.4 Progettazione

3.4.1 Introduzione

3.4.2 Le classi ‘*Target*’

3.4.3 Le classi ‘*Test*’

In questa sezione spiego come ho rappresentato i *test* da lanciare, partendo dai moduli che mi sono stati forniti e sui quali ho effettuato il *refactoring*. Questa parte riprende diversi concetti di §3.3, o per meglio dire ne rappresenta l'applicazione concreta all'interno del progetto.

3.4.4 Creazione ed esecuzione

In questa sezione spiego i servizi che si occupano della creazione e l'esecuzione delle classi di test. La creazione comprende una classe *factory* e un *pattern registry*; l'esecuzione comprende un *service* per l'esecuzione del singolo *test* e un altro per la gestione di più *test* eseguiti in serie. In merito a quest'ultimo parlerò anche della soluzione adottata per tenere traccia del *context* che permette ai *test* di prendere in *input* il risultato dei *test* eseguiti in precedenza.

3.4.5 Persistenza

In questa sezione spiego i servizi utilizzati per implementare uno storico dei *test* che conservi anche i risultati di quest'ultimi. Descriverò il *service* addetto alla persistenza e il *pattern repository* che interagisce con il *database SQLite*, motivando la scelta di quest'ultimo rispetto a un tradizionale *DBMS*.

3.4.6 API

In questa sezione spiego gli *endpoint* messi a disposizione dall'*API* implementata per accedere ai servizi del *framework*.

3.4.7 Frontend

In questa sezione spiego il ruolo del *frontend* come applicazione dimostrativa. È un punto un po' fuorviante, in quanto il senso del *frontend* è dimostrare l'estendibilità del progetto ad altre componenti che non abbiano diretto accesso alla logica interna del *framework*.

3.5 Programmazione

In questa sezione parlo della stesura del codice ed eventuali difficoltà/incoerenze rispetto alla progettazione che ho incontrato. Parto parlando prima dello sviluppo in *Python* del *framework* e successivamente mi soffermo sullo sviluppo del *firmware* in *C*.

3.6 Verifica e validazione

In questa sezione parlo dei controlli effettuati per verificare la correttezza del codice. Mi soffermo sul motivare perché si non ho puntato a un alta copertura di codice e su quali punti mi sono concentrato.

3.7 Resoconto del lavoro svolto

3.7.1 I risultati conseguiti

In questa sezione discuto i risultati raggiunti sul piano qualitativo che su quello quantitativo, specificando i prodotti *software* e di documentazione che sono risultati alla fine del progetto. Faccio poi una presentazione del progetto completo nel suo insieme, discutendone i diversi tipi di utilizzi che se ne possono fare dal punto di vista di un utente esterno.

3.7.2 Migliorie possibili

In questa sezione parlo di possibili migliorie che scoperto riguardando il progetto una volta finito. Cerco di dare una mia valutazione alle componenti del prodotto in relazione all'impegno che è servito per realizzarle, ne riconosco le limitazioni, spiego su cosa mi concentrerei se dovessi continuare a lavorarci. Discuto inoltre se vi sono scelte che farei diversamente se dovessi ricominciare da zero il progetto.

Capitolo 4

Valutazione retrospettiva

4.1 Valutazione del raggiungimento degli obiettivi

In questa sezione discuto i risultati ottenuti rispetto agli obiettivi prefissati, sia quelli del progetto che quelli personali.

4.2 La maturazione professionale

In questa sezione discuto l'esperienza lavorativa da me maturata e quali conoscenze ho acquisito in questo percorso.

4.3 Confronto con il piano di studi

In questa sezione discuto la differenza quali competenze del piano di studi mi hanno aiutato in quest'esperienza, quali mi mancavano e metto in evidenza alcune lacune de percorso di studi. Aggiungo delle mie opinioni sul piano di studi che ho sviluppato confrontandomi anche con alcuni colleghi in azienda e in relazione a quello che ho visto finora nelle mie esperienze lavorative.

Acronimi e abbreviazioni

API Application Programming Interface. [i](#), [20](#)

CAN Controller Area Network. [i](#), [20](#)

CI/CD Continuous Integration/Continuous Delivery. [i](#), [10](#)

CLI Command Line Interface. [i](#), [12](#)

CRA Cyber Resilience Act. [i](#), [16](#)

CSMS Cyber Security Management System. [i](#)

IDE Integrated Development Environment. [i](#), [13](#)

IoT Internet of Things. [i](#), [17](#)

MISRA Motor Industry Software Reliability Association. [i](#), [12](#)

OEM Original Equipment Manufacturer. [i](#), [7](#)

PO Product Owner. [i](#), [4](#)

PoC Proof of Concept. [i](#), [15](#)

R&D Research and Development. [i](#), [3](#)

UART Universal Asynchronous Receiver-Transmitter. [i](#), [29](#)

UDS Unified Diagnostic Services. [i](#), [27](#)

Glossario

Agile Fondato sui principi dell'*Agile Manifesto* (2001), promuove il coinvolgimento continuo del cliente, il rilascio frequente di valore funzionante e l'auto-organizzazione dei *team*. *Framework* come *Scrum* e *Kanban* ne rappresentano applicazioni pratiche diffuse. [i](#), [5](#)

API Interfaccia che mette a disposizione una serie di servizi tramite cui componenti *software* distinti possono comunicare e scambiarsi dati, senza necessità di conoscere i dettagli implementativi reciproci. [i](#)

CAN Protocollo di comunicazione seriale *multi-master*, utilizzato per la comunicazione tra ECU (*Electronic Control Units*) in ambito *automotive*. Le varie ECU sono collegate fisicamente tramite il *CAN bus*, e comunicano mandando messaggi *broadcast* nella rete condivisa. [i](#)

CI/CD (Continuous Integration/Continuous Delivery) Pratica *DevOps* che automatizza le fasi di integrazione, *test* e rilascio del *software*. La *Continuous Integration* prevede l'unione frequente del codice in un *repository* condiviso con *build* e *test* automatici, per individuare errori precocemente. La *Continuous Delivery/Deployment* estende il processo automatizzando il rilascio in produzione, riducendo tempi e rischi legati alle *release* manuali. [i](#)

CRA (Cyber Resilience Act) Regolamento dell'Unione Europea che introduce requisiti obbligatori di cybersicurezza per i prodotti con elementi digitali (*hardware* e *software*) immessi sul mercato europeo. Impone ai produttori obblighi come la gestione delle vulnerabilità, l'aggiornamento sicuro del *software* e la fornitura di informazioni chiare agli utenti sulla

sicurezza del prodotto, con l'obiettivo di ridurre il numero di dispositivi vulnerabili e responsabilizzare i produttori anche dopo la vendita.. [i](#)

Embedded Indica un sistema integrato *hardware-software* progettato per svolgere una funzione specifica e dedicata all'interno di un dispositivo. I sistemi *embedded* operano tipicamente con risorse limitate (CPU, memoria, energia) e in ambiti come: centraline auto, elettrodomestici, dispositivi IoT, *wearable* e strumentazione medica. [i](#), [1](#)

Functional Safety Ramo della sicurezza che si occupa di garantire il corretto comportamento di un sistema o uno strumento in seguito al cambiamento dei parametri di *input*, o al verificarsi di errori e/o guasti *hardware*. [i](#), [2](#)

IDE Ambiente di Sviluppo Integrato, *software* che riunisce in un'unica interfaccia gli strumenti necessari alla scrittura di codice. [i](#)

ISO-TP / ISO 15765-2 Protocollo di livello trasporto definito dallo *standard* ISO 15765-2, usato principalmente nel settore *automotive* per trasmettere messaggi su *bus* CAN. Poiché i *frame* CAN *standard* hanno un *payload* limitato a 8 *byte*, ISO-TP permette di segmentare messaggi più grandi in più *frame* e di riassemblarli a destinazione, gestendo anche il controllo di flusso tra trasmettitore e ricevitore. [i](#), [27](#)

Machine Learning (ML) Branca dell'intelligenza artificiale che studia algoritmi capaci di apprendere *pattern* dai dati, migliorando le proprie prestazioni su un compito senza essere esplicitamente programmati per eseguirlo. [i](#), [14](#)

MISRA Organizzazione che pubblica linee guida di programmazione (principalmente per C e C++) volte a migliorare sicurezza, affidabilità e portabilità del codice, soprattutto in ambito *automotive* ed *embedded/safety-critical*. Gli *standard* MISRA definiscono regole e *best practice* per evitare costrutti ambigui o pericolosi del linguaggio, e sono spesso richiesti in contesti normativi o di certificazione. [i](#)

Modbus Protocollo di comunicazione sviluppato originariamente per l'automazione industriale, basato su un'architettura *master/slave*. Permette a un dispositivo *master* di leggere e scrivere dati (registri, *coil*, ingressi digitali/analogici) su uno o più dispositivi *slave* tramite richieste e risposte strutturate in un formato semplice e aperto. [i](#), [28](#)

Open Source Prodotto *software* in cui il codice sorgente è reso pubblicamente disponibile, consentendo a chiunque di visualizzarlo, modificarlo e ridistribuirlo. [i](#), [11](#)

PoC *Proof of Concept*, si tratta di un prodotto che ha lo scopo di dimostrare la fattibilità di un progetto andando ad implementare, e mettendo in relazione, diverse tecnologie e idee sulle quali esso si basa. Può successivamente divenire una base iniziale sulla quale andare a sviluppare il progetto vero e proprio. [i](#)

Product Owner Figura chiave nel metodo *Agile*, responsabile di massimizzare il valore del prodotto sviluppato dal *team*. Definisce la visione del prodotto, gestisce e priorizza le attività, e traduce le esigenze di *stakeholder* e clienti in requisiti chiari per il *team*. I suoi compiti possono variare in base all'organizzazione della realtà in cui si trova, andando a sovrapporsi in parte con quelli del *Project Manager*. [i](#)

Scrum *Framework* che applica i principi *Agile* per la gestione e lo sviluppo di prodotti complessi, organizzato in cicli temporali brevi e a durata fissa chiamati *Sprint* (tipicamente 2-4 settimane). Una figura chiave è lo *scrum master*, responsabile di facilitare l'adozione e la corretta applicazione del *framework Scrum* all'interno del *team*. Egli rimuove ostacoli che rallentano il lavoro, facilita le cerimonie *Scrum* (*Daily*, *Sprint Planning*, *Review*, *Retrospective*) e isola il *team* da interferenze esterne, promuovendo il miglioramento continuo dei processi. [i](#), [6](#)

UART (Universal Asynchronous Receiver-Transmitter) Periferica *hardware* per la comunicazione seriale asincrona punto-punto, ampiamente usa-

ta nei sistemi *embedded*. Trasmette dati *bit* per *bit* su due linee separate, sincronizzandosi tramite *bit* di *start/stop* e una velocità di trasmissione (*baud rate*) concordata a priori tra i due dispositivi. [i](#)

UDS (Unified Diagnostic Services) Anche conosciuto come ISO 14229, è un protocollo a livello applicativo usato nel settore *automotive* per la comunicazione diagnostica tra un *tool* esterno e le centraline elettroniche (ECU) di un veicolo. Definisce un insieme di servizi identificati da codici esadecimali scambiati tramite richieste e risposte strutturate. UDS si appoggia tipicamente a protocolli di trasporto come ISO-TP (ISO 15765-2) per la trasmissione dei messaggi su *bus* CAN. [i](#)

Bibliografia

Siti

Cyber Resilience Act. URL: <https://digital-strategy.ec.europa.eu/en/policies/cyber-resilience-act>.

ETSI EN 303645. URL: https://www.etsi.org/deliver/etsi_en/303600_303699/303645/03.01.03_60/en_303645v030103p.pdf.